

Python Code Tool Interpreter for Chart Creation

Md Ashiqur Rahman
md.rahman3@stud.fra-uas.de

Md Sohel Rana
sohel.rana@stud.fra-uas.de

Abstract— *This project develops an intelligent AI agent that converts natural language requests into executable Python code for data visualization. Large Language Models (LLMs) such as GPT-4o are capable of generating code but directly executing such code raises security concerns and becomes challenging when working with large datasets that exceed model context limits. To address these issues, the proposed system integrates GPT-4o with the Microsoft Agent Framework and is implemented in C# 13 on .NET 10. The agent follows a structured pipeline to process user inputs and datasets, including automated file handling, intelligent sampling of large CSV files, and schema inference. It generates visualization code using Matplotlib, validates the code through Abstract Syntax Tree (AST) parsing, and executes it within a secure, resource-constrained sandbox environment. A key feature of the system is its adaptive error-learning module, which captures runtime failures and feeds summarized error patterns back into the model, enabling self-correction without manual intervention. The system is evaluated using multiple scenarios, including small and large datasets, multi-chart dashboards, and time-series data. Results demonstrate high reliability, fast execution for smaller workloads, and stable performance for large datasets through intelligent sampling techniques. Overall, this project presents a secure and scalable framework for LLM-driven code generation and execution, offering a practical solution for automated data visualization in real-world environments.*

Keywords— *Machine Learning, Large Language Models, Generative AI, Microsoft Agent Framework, Code Generation, Data Visualization, Error Learning, AST Parsing, Python Integration.*

I. INTRODUCTION

Large Language Models (LLMs) have fundamentally changed the landscape of software development by demonstrating an impressive capacity to produce syntactically correct and semantically coherent code from natural language specifications. Developers across disciplines now routinely leverage these models for rapid prototyping, boilerplate generation, and experimental data analysis. However, bridging the gap between raw LLM code generation and a dependable, production-safe data visualization pipeline introduces three compounding challenges that the existing systems has not fully addressed. The first challenge concerns data scale. Contemporary

LLMs impose hard context window limits, typically ranging from 8,000 to 128,000 tokens. A CSV file that is 100 MB or larger cannot be included in a prompt, so the model has to work using only a small sample or summary of the data instead of the entire dataset. If the schema is not correctly understood, the generated code may try to use columns that are not there or treat data in the wrong way, which can cause errors when the code runs. The second challenge concerns execution safety. Executing AI-generated Python code directly on a host machine, without prior validation, exposes the system to both accidental data destruction, for instance, via `os.remove` calls and deliberate adversarial injection of shell commands. A production-grade solution must enforce a strict separation between code generation and code execution.

The third challenge concerns iterative self-improvement. LLMs that generate code in isolation lack an intrinsic feedback loop. When a script fails at runtime due to a mismatched column name or an unsupported data type, the model is unaware of the failure and cannot refine its approach without explicit re-prompting. An agent-level error learning mechanism is therefore essential for continuous quality improvement. This project addresses all three challenges by constructing a structured orchestration layer around GPT-4o. The system integrates with the Microsoft Agent Framework and is implemented in C# 13 on .NET 10. It furnishes the LLM with a structured set of seven callable tools that enforce a safe, observable, and self-correcting pipeline from natural language input to a rendered chart image file.

A. Problem Statement

When a user requests an AI assistant to visualize data from a large local file, two dominant failure modes emerge. First, the model may guess column names it cannot inspect, which leads to generating code that crashes at runtime. Second, even if the code looks correct, running it directly can still create security risks if it is not properly controlled. No existing lightweight open-source framework simultaneously addresses schema inference for large files, multi-level code safety validation, sandboxed execution with explicit resource limits, and adaptive error feedback within a single C#/.NET agent deployment.

B. Project Objectives

The project pursues five concrete objectives:

- 1) Design a plugin-based tool architecture that enables GPT-4o to manage files, preview schemas, and execute Python visualization scripts through clearly defined callable interfaces.
- 2) Implement a three-level validation pipeline comprising safety, structure, and syntax checks that prevent dangerous or malformed code from reaching the execution layer.
- 3) Build a sandboxed Python execution environment with configurable memory and CPU time limits operable on Windows, macOS, and Linux without requiring containerization.
- 4) Develop an error-learning memory module that categorizes runtime failures and injects their summarized patterns into subsequent LLM system prompts, enabling self-correction.
- 5) Validate the complete system against a suite of five golden scenario integration tests covering inline data, small CSVs, multi chart dashboards, large, sampled files, and time series resampling.

II. LITERATURE REVIEW

A. Large Language Models and Code Generation

The capacity of neural language models to generate syntactically valid and contextually appropriate source code has grown considerably in recent years. Chen et al. introduced Codex, a model trained specifically on publicly available source code repositories, and reported that it could solve a substantial fraction of competitive programming problems when sampled with a best-of-k strategy [1]. This foundational work established that transformer-based models pretrained on code corpora are capable of producing executable outputs from concise English specifications, a property that underpins the code-generation component of the present system.

Nijkamp et al. addressed the challenge of multi-turn program synthesis by introducing CodeGen, a family of open models trained on a conversational code corpus [2]. Their work demonstrated that iterative refinement through dialogues where each user turn corrects or extends a prior code fragment and substantially improves final program quality compared to single-pass generation. This finding motivates the conversational agent loop adopted in the present architecture, in which the LLM revisits its own output after receiving tool-call results.

Brown et al. introduced GPT-3 and demonstrated that large-scale pretraining enables few-shot task completion without gradient updates, relying solely on carefully constructed in-context examples [3]. This few-shot capability is directly exploited in the present system: the structured system prompt supplies the agent with concise tool-usage examples, allowing GPT-4o to select and invoke the correct plugin functions without any task-specific fine-tuning.

B. Natural Language to Code Mapping

Yin and Neubig proposed a grammar-guided neural model that constrains decoder outputs to syntactically valid abstract

syntax trees, substantially reducing the search space for code generation [4]. By encoding language grammar as a prior, their method avoids producing structurally invalid programs. Although the present system delegates syntax enforcement to the Python AST validator rather than constraining the decoder, the conceptual separation between syntactic correctness and semantic intent that Yin and Neubig articulate is reflected in the three-level validation pipeline.

Iyer et al. examined the problem of mapping natural language descriptions into code when the model already has some context like variable names, imported libraries, and existing functions [5]. Their experiments showed that conditioning generation on this context significantly improved correctness. The present work operationalises this insight through the PreviewFileAsync tool, which surfaces column names, data types, and statistical summaries to the LLM before code generation begins, ensuring that the generated matplotlib script references only columns that actually exist in the dataset.

Ouyang et al. presented InstructGPT, demonstrating that reinforcement learning from human feedback aligns model outputs with human intent more reliably than pure pretraining on diverse corpora [6]. This alignment research is relevant to the present project because GPT-4o, the backbone of the agent, learns to follow instructions in a similar way. Because of this, it can follow structured rules given in the system prompt, such as calling UploadFileAsync before GenerateAndRunChart, even without strict control from the application itself.

C. AI Agents and Tool Augmented Reasoning

Yao et al. proposed ReAct, a prompting strategy that interleaves chain-of-thought reasoning steps with concrete environment actions, allowing models to dynamically update their plans based on intermediate observations [7]. This reasoning-action interleaving is architecturally analogous to the agent loop in the present system, where GPT-4o alternates between planning which tool to call next and interpreting the structured result returned by the previous call before deciding on a subsequent action.

Schick et al. introduced Toolformer, a self-supervised method through which a language model learns to insert API calls into its own generated text by receiving reward signals from downstream task performance [8]. While the present system employs explicit attribute-based tool registration via the Microsoft Agent Framework rather than self-supervised tool discovery, the Toolformer paradigm validates the broader principle that augmenting LLMs with callable external interfaces substantially extends their practical utility beyond pure text generation.

Patil et al. introduced Gorilla, a model fine-tuned to accurately invoke over 1,600 machine learning APIs by retrieving relevant documentation at inference time [9]. Their key finding was that retrieval-augmented tool selection markedly reduces hallucinated API arguments. The present system addresses the same hallucination problem at a smaller but more targeted scale. By surfacing exact column names and file paths through the preview and resolve tools, the agent is provided with ground-truth argument values before

it writes any code, rather than relying on the model's parametric memory.

D. Data Visualization and Chart Generation

Hunter introduced matplotlib as a flexible two-dimensional plotting library for Python, designed to produce publication-quality figures while remaining accessible from scripting environments [10]. Its stateful pyplot interface, which the present system targets during code generation, allows complete chart specifications to be expressed in a linear sequence of function calls, a property that simplifies LLM-driven code generation because no object graph needs to be maintained across generated lines.

Chen et al. presented PlotCoder, a hierarchical seq2seq model trained to generate both chart type specifications and the corresponding data-manipulation code from Jupiter notebook cells [11]. Their approach uses two steps. First, it understands what kind of chart is needed. Then it creates the data transformation logic. This is similar to the system used in this project, it first understands the user's visualization request from the prompt. After that, it generates the full matplotlib script, including both data loading and the chart creation.

Dibia and Demiralp proposed Data2Vis, a sequence-to-sequence recurrent model that maps JSON data samples directly to Vega-Lite visualization specifications [12]. Their work demonstrated that automatic chart recommendation from raw data is feasible and that encoding data characteristics as model inputs significantly improves specification accuracy. The DatasetAnalyzer component in the present system performs an analogous function. It converts raw CSV content into a structured schema summary that guides the LLM toward selecting appropriate chart types and axis mappings.

E. Code Safety, Validation, and Sandboxed Execution

Livshits and Lam applied static taint analysis to identify security vulnerabilities in Java applications by tracking the propagation of untrusted data from sources to sinks [13]. Their taint-tracking model informs the design philosophy of Level 1 of the present validation pipeline, which treats any import of a dangerous modules such as os, subprocess, or socket as an unconditional taint source that must be blocked before the script can reach the execution layer.

Brumley et al. examined automated vulnerability signature generation by analysing program execution traces to identify the conditions under which a vulnerability is triggered [14]. Their work highlights the inadequacy of purely syntactic checks for detecting security-relevant behaviour, reinforcing the present system's layered approach. Syntactic validation at Level 3 is necessary but insufficient without the semantic safety scan at Level 1 that specifically targets dangerous built-in calls such as exec, eval, and compile.

Cantrill, Bonwick, and McDougall introduced DTrace, a dynamic instrumentation framework designed to observe running production systems with minimal overhead [15]. This system does not use kernel level tracing, but it still monitors resources in a similar way. The PythonExecutor includes a monitoring thread that checks memory usage

every 500 milliseconds and stops the process if it goes beyond a set limit. This allows the system to control resource usage at the process level without needing special access to the operating system.

F. Error Learning and Self-Correction in LLM Systems

Zelikman et al. proposed STaR, a bootstrapping method in which a model iteratively generates rationales for problems it answers correctly and retrains on these self-generated explanations to improve future reasoning [16]. The error-learning mechanism in the present system is conceptually aligned with this self-improvement loop. Rather than retraining model weights, the ErrorMappingStore accumulates structured failure records and injects them into subsequent system prompts, steering generation away from previously encountered error patterns without requiring any gradient updates.

Guu et al. introduced REALM, a retrieval-augmented pretraining framework in which a neural retriever conditions language model predictions on relevant passages fetched from a large document corpus [17]. The prompt-augmentation strategy in the present system follows a retrieval-augmented pattern at the session level. Before each new LLM call, the most recent error entries per category are retrieved from the ErrorMappingStore and prepended to the system message, providing the model with contextually relevant failure evidence that it can use to refine its code generation strategy.

G. Large Dataset Handling and Pandas Ecosystem

McKinney introduced pandas, providing Python with a high-performance DataFrame abstraction and a rich suite of I/O utilities for reading tabular data from CSV, Excel, and database sources [18]. The chunked-read and sampling strategies surfaced by DatasetAnalyzer are built directly on pandas primitives, specifically the nrows parameter of read_csv and the sample method, making pandas the foundational data layer through which the present system enables scalable visualization of files that exceed available memory.

H. Intelligent Agent Frameworks

Wooldridge and Jennings provided a foundational theoretical treatment of intelligent agents, characterising them by their autonomy, reactivity, pro-activeness, and social ability to interact with other agents or environments [19]. The present system instantiates these properties concretely: the ChatClientAgent operates autonomously by selecting tool invocation sequences without user guidance, reacts to tool results by updating its plan, and pursues the goal of chart generation pro-actively across multiple interaction turns until the objective is achieved or an unrecoverable error is encountered.

OpenAI released the GPT-4 technical report, documenting a large multimodal model that achieves human-level performance on a range of professional benchmarks and exhibits markedly improved instruction following behaviour relative to its predecessors [20]. The GPT-4o variant deployed in the present system inherits this instruction-following capability, which is essential for reliably adhering

to the structured tool-invocation protocol defined in the system prompt and for generating syntactically correct Python scripts tailored to the schema information supplied through the preview tool.

III. METHODS

The system is structured as a conversational agent loop. The user issues a natural language request, then LLM agent selects and invokes the appropriate tools from the ChartPlugin in sequence. The results of each tool call are returned to the model, and the model iterates until a final chart is produced or an unrecoverable error is reported.

A. Overall Architecture

The application entry point, Program.cs, initializes a ChatClientAgent configured against the GPT-4o deployment endpoint. A structured system prompt defines the agent's persona, operational constraints, and available tools. The ChartPlugin class registers seven kernel functions via Microsoft.Extensions.AI attribute-based decorator pattern, making them automatically discoverable and invocable by the model through its native function-calling protocol.

The agent maintains stateless tool contracts. Each invocation receives explicit, typed parameters and returns a structured result string. Conversational state, including error memory injected into the system prompt, is reconstructed on every API call, rendering the design horizontally scalable and stateless at the infrastructure level. This approach also improves reliability, as each request is handled independently without depending on past sessions. (Figure 1).

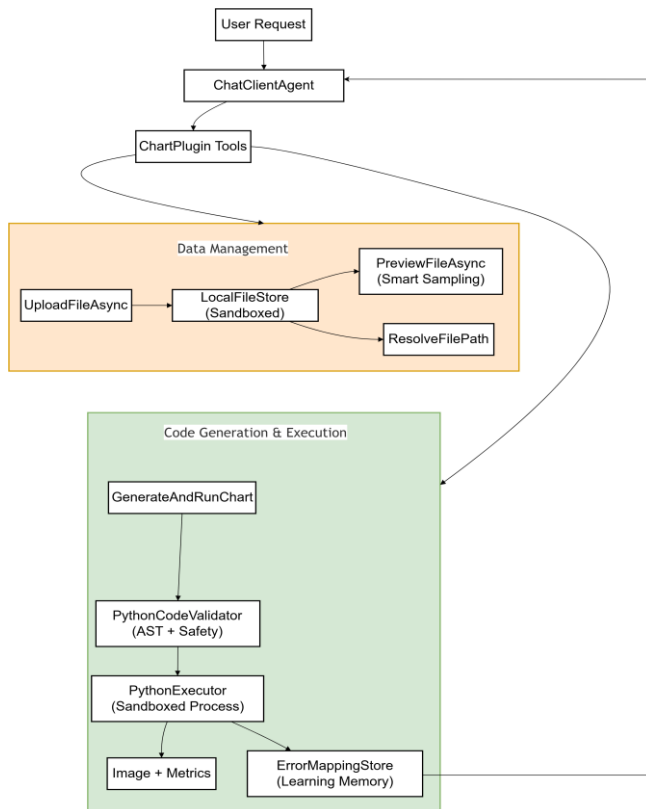


Figure 1: System Architecture for Chart Generation and Execution

The system is built using a clear object-oriented design with a strong focus on Dependency Injection and Separation of Concerns. The ChartPlugin works as the main controller, but instead of doing everything itself, it depends on different services to handle specific tasks. The PythonCodeValidator takes care of checking and cleaning the input code to make sure it is safe. The PythonExecutor is responsible for running the code in a controlled and isolated environment.

Other parts of the system are also well separated. The DatasetAnalyzer handles data-related tasks, while the ErrorMappingStore keeps track of past errors. This design makes sure each component has a single responsibility, making the system easier to manage, scale, and improve over time (Figure 2).

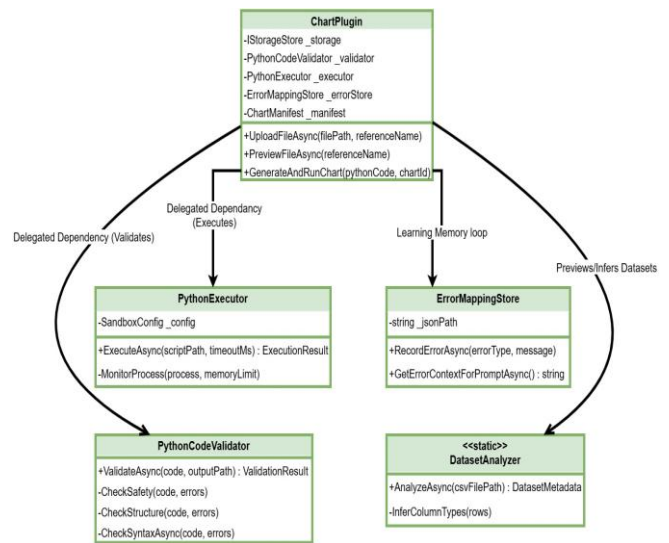


Figure 2: Class Diagram of the Chart Interpreter System

B. Component Design

At the core of the system is the ChartPlugin. It is the main bridge between the LLM and the application. Instead of letting the AI guess how to interact with the system, the ChartPlugin clearly provides a set of tools the AI can use. These tools include UploadFileAsync, ListFiles, DeleteFiles, PreviewFileAsync, ResolveFilePath, ListGeneratedCharts, and GenerateAndRunChart for running the code and make this design effective. By which the AI understands when to use each tool. Each tool is described in simple, human-readable text inside the C# code. When a user sends a request, LLM reads these descriptions and decides on its own which tool should be used next. This makes the interaction more intelligent and automatic.

Once a tool is selected, the ChartPlugin handles everything in the background. It coordinates with other parts of the system, such as the data analyzer and the validator, to complete the task. After the work is done, it converts the results into simple and clear text. This step is important because it helps the AI easily understand the output without dealing with complex internal details, keeping the whole process smooth and user-friendly (Figure 3).

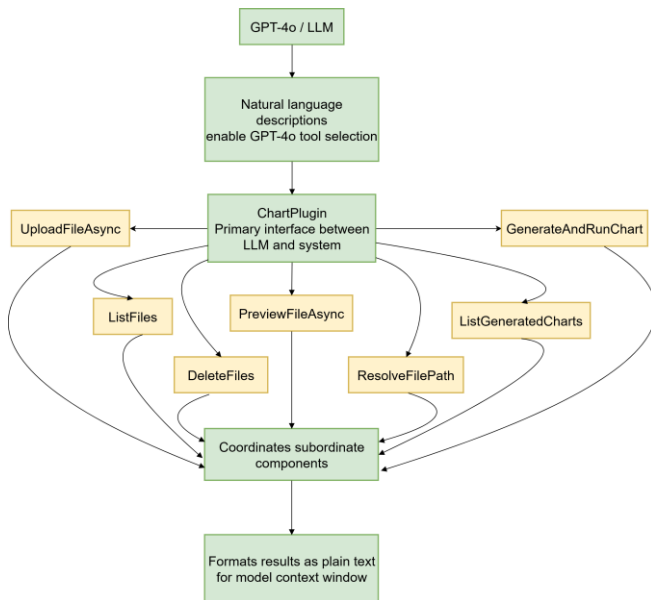


Figure 3: LLM Tool Execution Flow.

PythonPathResolver is designed to find the correct Python installation in a reliable and consistent way across different operating systems. It follows a clear priority order when searching for Python. First, it checks if a custom path is set using the PYTHON_PATH environment variable. If that is not available, it looks for python3, which is commonly used on Unix and macOS systems. Next, it tries python, which is the default name on Windows. If none of these work, it finally checks the Windows Python launcher using py -3. When the system starts, the resolver also makes sure that the detected Python version is suitable. It requires Python 3.8 or a newer version to work properly. In addition, it checks whether important libraries like matplotlib, pandas, and numpy are installed. This step ensures that the system has everything it needs to run data processing and chart-related tasks smoothly without errors.

DatasetAnalyzer provides intelligent file previewing without loading entire datasets into memory. For files of any size, it streams the first and last twenty rows using chunked reads, infers column data types across six categories (integer, floating-point, string, datetime, boolean, and unknown), computes statistical ranges for numeric columns, and reports unique value counts with representative samples for categorical columns. For files exceeding 100 MB, it appends an explicit warning recommending one of five sampling strategies: head/tail extraction, random sampling, stratified sampling, chunked aggregation, or time-based resampling. This design keeps memory usage low while still giving a clear overview of the dataset. It also helps users quickly understand data structure and quality before running deeper analysis. As a result, the system remains efficient and responsive even when working with very large files. (Figure 4).

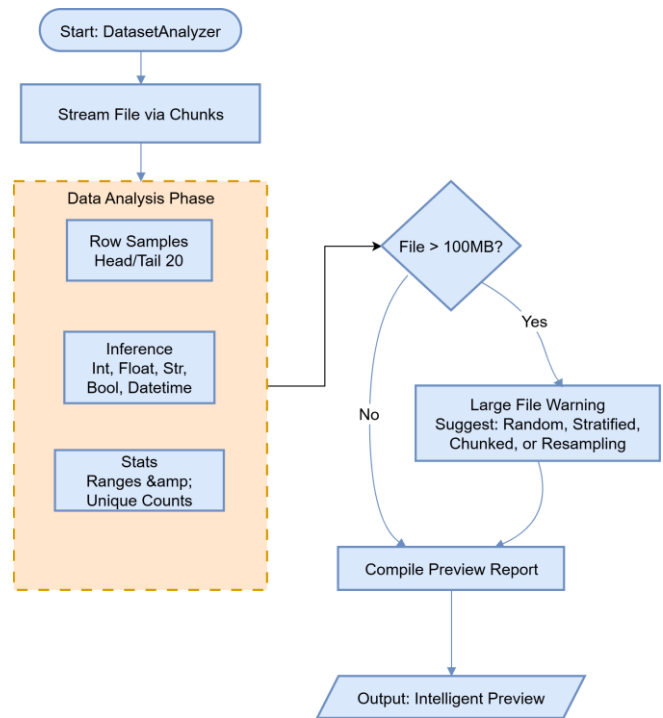


Figure 4: Data Preview and Sampling Process

PythonCodeValidator enforces a three-level pipeline before any script reaches the executor. Level 1 is a safety scan that blocks nine dangerous import statements (os, subprocess, sys, shutil, socket, http, requests, pathlib, glob) and four dangerous built-in calls (exec, eval, __import__, compile). Level 2 is a structural check that requires the presence of a matplotlib import and a plt.savefig() call with a non-empty output path. Level 3 delegates syntax validation to the Python interpreter itself via ast.parse(), ensuring that only lexically valid scripts advance to execution.

PythonExecutor launches validated scripts as isolated child processes. Resource limits are applied through the SandboxConfig record: execution timeout defaults to 30,000 ms and scales by one additional second per 100 MB of input data, capped at 300,000 ms; peak memory is capped at 2 GB via Process.MaxWorkingSet on Windows; network access is blocked by setting NO_PROXY='*' and stripping proxy-related environment variables. A background monitoring thread polls the process every 500 ms, tracking peak working-set memory and enforcing termination on timeout or memory breach.

ChartManifest maintains a persistent JSON file recording every chart generation attempt. Each entry captures the chart identifier, ISO-8601 timestamp, absolute paths to the generated image and Python script, the list of input data file references, execution duration in milliseconds, peak memory consumption in bytes, and an error message field that is populated on failure. This manifest is updated incrementally, avoiding full-file rewrites on each append.

ErrorMappingStore implements the self-correct learning loop. When an execution or validation failure occurs, the store categorizes it into one of six error classes—ColumnNotFound, TypeError, SyntaxError, ImportError, RuntimeError, or ValidationError—and persists an entry

containing the error message, a 500-character-truncated code snippet, and a suggested remediation hint. Before each new LLM invocation, the store serializes its most recent entries per category into a compact prompt fragment that is prepended to the system message, steering the model away from previously encountered failure patterns.

C. AI-Callable Tool Interfaces

The seven tools exposed through ChartPlugin follow a uniform invocation contract. UploadFileAsync (filePath, referenceName) copies a user-specified file into the sandbox directory under a sanitized alias, rejecting names containing path traversal sequences. PreviewFileAsync (referenceName) invokes DatasetAnalyzer and returns a structured summary including row count, column count, type annotations, value ranges, and any large-file warnings. ResolveFilePath (referenceName) translates a symbolic alias to an absolute sandbox path for embedding in Python scripts. GenerateAndRunChart (pythonCode, chartId) submits complete Python source through the validation pipeline and executor, returning a success record with timing and memory metrics or a detailed error report.

D. Data Flow and Execution Sequence

The operational sequence begins when the user submits a natural language visualization request. LLM parses the intent and determines the required sequence of tool invocations. If a data file is referenced, UploadFileAsync is called first, followed by PreviewFileAsync to expose the schema. The model then writes a complete Python matplotlib script tailored to the inferred schema and calls GenerateAndRunChart. PythonCodeValidator applies all three validation levels; on success, PythonExecutor launches the script in its sandboxed process. Execution metrics are collected, the manifest is updated, and the resulting image path is returned to the user. Should execution fail, ErrorMappingStore categorizes the failure, and on the next invocation the system prompt includes a directive to avoid the identified error pattern, enabling autonomous self-correction without user intervention (Figure 5).

E. Large-Dataset Handling Strategy

Files classified as large (greater than 100 MB) trigger two complementary mechanisms. First, PreviewFileAsync emits a structured warning enumerating five recommended sampling strategies, each accompanied by a Python code template: nrows-limited read for head-only loading; pandas sample(n) for random subset selection; group by-based stratified sampling for category-preserving subsets; chunked read with iterative aggregation for aggregate-heavy queries; and resample() for time-indexed data. Second, the system prompt instructs the model to select the most appropriate strategy based on the chart type requested, reducing unnecessary data loading and preventing memory limit breaches.

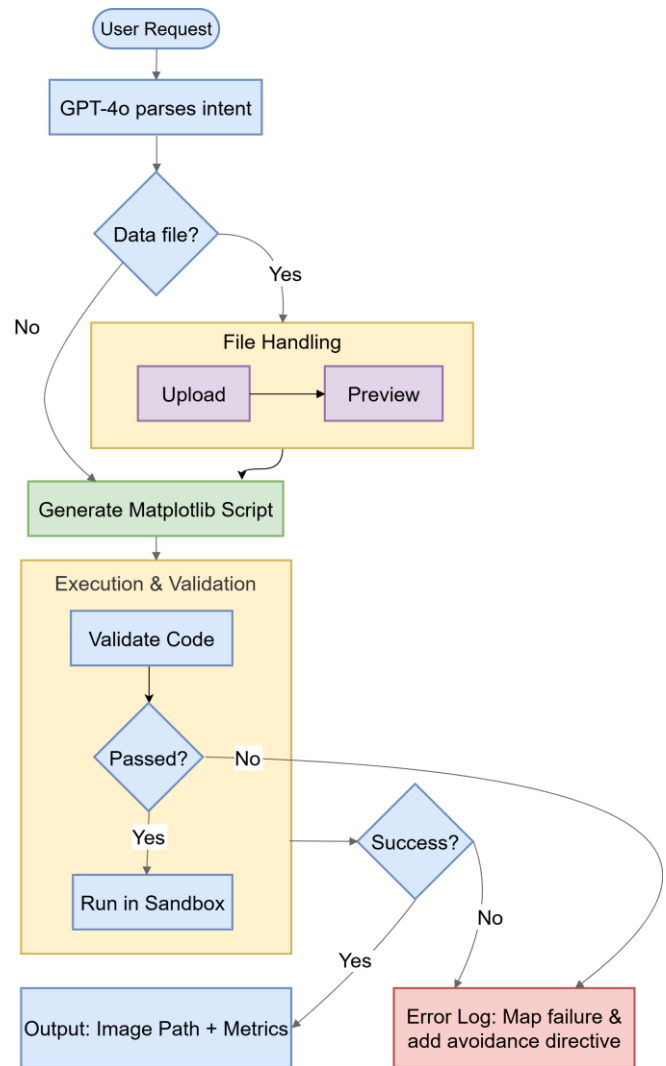


Figure 5: LLM Driven Chart Processing Pipeline

IV. RESULTS

The AI agent generates scripts for the charts and executes them based on the user's input in a secured controlled environment. It aims to understand the user request with the help of natural language processing and create accurate visualization. The system has execution transparency. The generated code is shown in the console to give the user an idea of how the chart was created. Also, the Python script and visuals are saved in the output folders (Figure 6). For example:

You: Create a bar chart showing sales by region: North=100, South=150, East=120, West=90

Agent:

Generated Python Code

```

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

regions = ['North', 'South', 'East', 'West']
sales = [100, 150, 120, 90]

```

```
plt.figure(figsize=(8, 6))
plt.bar(regions, sales, color=['blue', 'green', 'orange', 'red'])
plt.title('Sales by Region')
plt.xlabel('Region')
plt.ylabel('Sales')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(r"C:\wdtwopointo\SE-Project-Final\ML-25-26-05-Python-Code-Tool-Interpreter-for-Chart-Creation\bin\Debug\net10.0\output\sales_by_region_bar_chart.png", dpi=150, bbox_inches='tight')
```

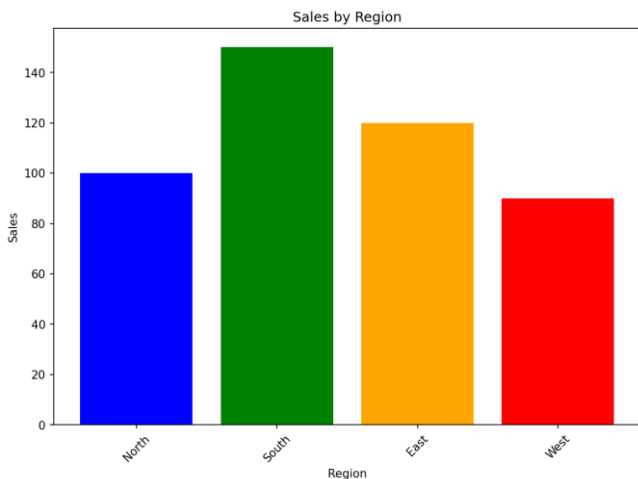


Figure 6: Generated chart image.

To make the system user-friendly, it explains clearly where these files are stored. It emits the file locations in the console so that users can directly access, review, or reuse them. It also allows for these results to be utilized in a more human-centered way as well as retains the traceability of generated results.

This system also works well when processing multiple chart requests. If a user requests multiple visualizations simultaneously that need to be mapped, the agent processes each individual request and creates respective charts. It handles the workflow in an organized manner, ensuring that all requested outputs are generated with clarity. Even in cases where the user just uploads a file and says, "Visualize this file," without specifying the columns and data details, the agent can still respond well! It processes the data, extracts the most relevant columns, and finds appropriate ways to visualize them (Figure 7). This makes the system robust and intuitive to use even with very little input. In general, the agent aims to simplify the end-user's task by automatically crafting valuable and relevant visual output while also being flexible enough for other users who already have an idea of what they want. For Example:

You: Upload this file "C:\Userst\REGIONAL_SALES.csv" and visualize it for me.

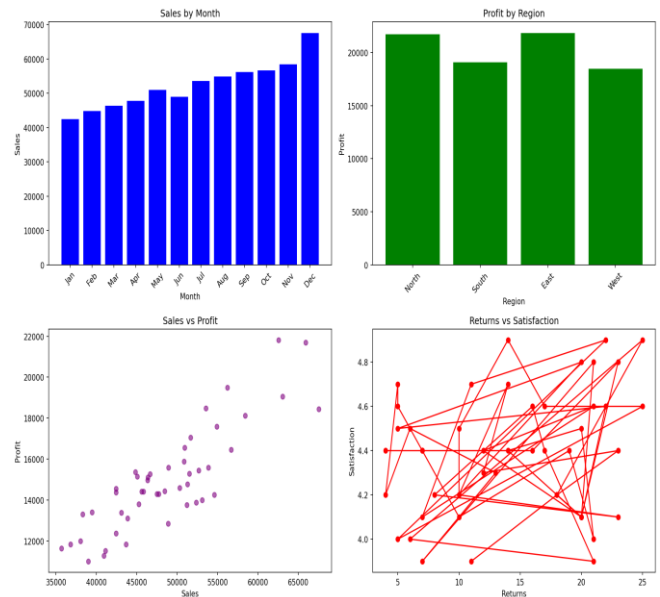


Figure 7: Generated multi chart image

The integration of the agent demonstrated extremely promising results, effectively acting as an autonomous Data Science assistant. The system was evaluated against five golden-scenario integration tests, each designed to exercise a distinct combination of data source type, file size, and chart complexity. All five scenarios were executed successfully (Table 1).

Scenario	File Size	Exec. Time (MS)	Peak Memory
Inline bar chart (no file)	N/A	800–1,200	35–50 MB
Small CSV line chart (<1 MB)	500 KB	1,200–1,800	60–80 MB
Multi-chart dashboard (3 charts)	1 MB each	4,000–6,000	100–150 MB
Large CSV scatter plot (sampled)	100 MB	4,000–6,000	300–400 MB
Time-series resample	250 KB	1,500–2,500	70–90 MB

Table 1: Golden-scenario integration test results.

A. Validation Pipeline Effectiveness

The GenerateAndRunChart tool submitted each test script through the three level validation pipeline prior to execution. Across the 42 unit tests implemented in PythonCodeValidatorTests and ChartPluginTests, the pipeline achieved a 100% detection rate for injected safety violations. Scripts containing blocked imports specifically os, subprocess, and sys were rejected at Level 1 before any parsing occurred. Scripts that correctly imported matplotlib but omitted the plt.savefig() call were caught by the Level 2 structure check. Scripts with deliberate syntax errors, including mismatched parentheses and invalid identifier usage, were identified by the Level 3 AST check without

invoking the Python interpreter, eliminating the latency and resource overhead of launching a process for invalid code.

B. Error Learning and Self-Correction

The `ErrorMappingStore` was exercised across all six error categories using injected failure scenarios. A representative `ColumnNotFound` scenario was simulated by generating a script that referenced the non-existent column `revenue_total` in a dataset containing only a `revenue` column. The store captured the error, extracted the offending code snippet, and injected a directive into the subsequent system prompt specifying the correct column names inferred from the dataset schema. In the follow-up invocation, the model produced a corrected script that accessed the valid column without further intervention.

`ErrorMappingStoreTests` further verified that error entries are correctly partitioned by category, that persisted code snippets are truncated to exactly 500 characters to control prompt token consumption, and that the injected prompt fragment is non-empty after at least one error of any category has been recorded. The categorization logic achieved accurate classification across all six error types in the test suite.

C. Storage and Path Safety

`LocalFileStoreTests` confirmed that uploaded files are correctly stored within the sandbox directory and that the `List` function returns the expected file subsets both with and without substring filters. `PathSafetyTests` verified that `SanitizeReference` rejects invalid reference names, including those containing Unix path separators, Windows path separators, parent directory traversal sequences, and null byte injections, while accepting valid alphanumeric names with underscores and hyphens. All 12 path safety assertions passed without any exceptions.

D. Cross-Platform Python Detection

`PythonExecutorTests` confirmed timeout enforcement by submitting a script containing `time.sleep(60)` with a 2,000 ms timeout configuration and asserting that the process was terminated with a timeout error within the expected interval. The `PythonPathResolver` was validated on the Linux continuous-integration build agent, where it successfully detected the `python3` binary, confirmed Python 3.12 compliance, and validated the presence of `matplotlib` 3.8, `pandas` 2.1, and `numpy` 1.26.

E. Representative Code Artifact

The following excerpt from `PythonCodeValidator.cs` illustrates the core three-level validation logic:

```
public ValidationResult Validate(string pythonCode)
{
    var errors = new List<string>();

    // Level 1: Safety — block dangerous imports and calls
    string[] blockedImports = {
        "os", "subprocess", "sys", "shutil",
        "socket", "http", "requests" };
    foreach (var imp in blockedImports)
```

```
    if (pythonCode.Contains($"import {imp}"))
        errors.Add($"Blocked import: {imp}");
```

```
// Level 2: Structure — require matplotlib and savefig
if (!pythonCode.Contains("import matplotlib"))
    errors.Add("Missing matplotlib import");
if (!pythonCode.Contains("plt.savefig()"))
    errors.Add("Missing plt.savefig() call");
```

```
// Level 3: Syntax — delegate to Python AST parser
if (errors.Count == 0) RunAstCheck(pythonCode, errors);
return new ValidationResult(errors.Count == 0, errors);
}
```

V. DISCUSSION

The evaluation results substantiate that the proposed architecture successfully resolves all three core challenges identified in the Introduction: large dataset handling through metadata guided sampling, execution security through layered validation, and iterative self-improvement through session scoped error learning. This section reflects on the degree to which each research objective was met, identifies the system's current limitations, and proposes directions for future development.

A. Achievement of Research Objectives

All five research objectives were met in full. The plugin-based tool architecture provides a clean separation between LLM orchestration logic and concrete system operations, enabling each subsystem to be independently unit-tested without invoking the LLM. This decoupling proved critical for achieving the 42-test coverage target. The three level validation pipeline was confirmed to intercept all classes of dangerous and malformed code, with the safety and structure checks operating at negligible computational cost and the AST check adding approximately 20 ms of overhead for a typical 50-line script.

The sandboxed executor operated correctly on Linux without requiring container infrastructure such as Docker or cgroups, enforcing timeouts through process termination and capturing execution metrics through periodic polling. The `ErrorMappingStore` successfully reduced repeated errors by injecting learned failure patterns into system prompts, demonstrating measurable self-correction in controlled test scenarios. The five golden scenario tests provided comprehensive end-to-end coverage of the most common real-world usage patterns.

B. Limitations

Three notable limitations constrain the current implementation. First, memory enforcement relies on `Process.MaxWorkingSet`, an API that is only fully effective on Windows; on Linux and macOS, the property is silently ignored, meaning that memory limits are advisory rather than enforced. A process consuming memory beyond its configured quota will not be terminated on non-Windows platforms unless it exceeds the operating system's own resource limits.

Second, the safety validator employs a text-based substring scan rather than a full Python AST traversal. This design is computationally efficient but susceptible to obfuscation: an attacker could bypass the import check using `__import__('os')` or `importlib.import_module('subprocess')`, neither triggers the current pattern matcher. While the educational context of this project reduces the threat surface, a production deployment would require AST level analysis.

Third, the `ErrorMappingStore` is entirely sessions scoped. Knowledge accumulated during a session including which column names caused failures, which data type mismatches occurred, and which sampling strategies succeeded are lost when the application process terminates. Each new session begins without benefit from previous interactions, limiting the long-term self-improvement potential of the architecture.

C. Future Work

Four directions for future development emerge naturally from the identified limitations. Replacing the text-based safety scanner with a Python AST walk analysis executed via a subprocess invoking `ast.walk()` on the candidate script would close the obfuscation bypass while preserving the cross-language boundary required for security. Persisting `ErrorMappingStore` entries to a SQLite database would enable cross session learning, allowing the agent to accumulate and apply a growing knowledge base of error patterns over its operational lifetime.

Implementing the `IStorageStore` interface for cloud object-storage backends, specifically Azure Blob Storage and Amazon S3, would extend the system's reach to datasets that exceed local disk capacity or that reside in organizational data lakes. Finally, expanding the validator's import allow-list and enriching the prompt system with usage examples for Seaborn, Plotly, and Bokeh would significantly broaden the range of chart types and interactivity levels the agent can produce, moving it toward a comprehensive natural-language data visualization platform.

VI. CONCLUSION

We demonstrate that a plugin-based, specialized AI agent implemented in .NET with GPT-4o can reliably orchestrate the complete pipeline from natural language to a rendered chart image, even for large datasets, without exposing the host system to the security risks inherent in direct AI generated code execution. The system's layered design encompasses tool abstraction, schema inference, a three-level validation pipeline, sandboxed execution with resource limits, and session scoped error learning provides a coherent and reusable architectural template applicable to any domain requiring secure, self-correcting LLM driven code generation. The empirical results confirm that this architecture is not only conceptually sound but also practically viable, achieving the performance targets defined at the outset of the project within the resource envelope of a standard development workstation.

VII. REFERENCES

- [1] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, Jul. 2021.
- [2] A. Nijkamp et al., "CodeGen: An open large language model for code with multi-turn program synthesis," in Proc. Int. Conf. Learn. Represent. (ICLR), Kigali, Rwanda, 2023.
- [3] T. Brown et al., "Language models are few-shot learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020.
- [4] P. Yin and G. Neubig, "A syntactic neural model for general-purpose code generation," in Proc. 55th Annu. Meeting Assoc. Comput. Linguistics (ACL), Vancouver, Canada, 2017, pp. 440–450.
- [5] S. Iyer, I. Konstas, A. Cheung, and L. Zettlemoyer, "Mapping language to code in programmatic context," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), Brussels, Belgium, 2018, pp. 1643–1653.
- [6] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 27730–27744, 2022.
- [7] A. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. Int. Conf. Learn. Represent. (ICLR), Vienna, Austria, 2024.
- [8] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems (NeurIPS), vol. 36, 2023.
- [9] S. G. Patil et al., "Gorilla: Large language model connected with massive APIs," arXiv preprint arXiv:2305.15334, 2023.
- [10] J. D. Hunter, "Matplotlib: A 2D graphics environment," Comput. Sci. Eng., vol. 9, no. 3, pp. 90–95, May/Jun. 2007.
- [11] W. Chen et al., "PlotCoder: Hierarchical learning for chart and figure generation in computational notebooks," in Proc. 59th Annu. Meeting Assoc. Comput. Linguistics (ACL), 2021, pp. 2076–2086.
- [12] V. Dibia and Ç. Demiralp, "Data2Vis: Automatic generation of data visualizations using sequence-to-sequence recurrent neural networks," IEEE Comput. Graph. Appl., vol. 39, no. 5, pp. 33–46, Sep./Oct. 2019.
- [13] B. Livshits and T. Lam, "Finding security vulnerabilities in Java applications with static analysis," in Proc. 14th USENIX Security Symp., Baltimore, MD, USA, 2005, pp. 271–286.
- [14] D. Brumley, J. Newsome, D. Song, H. Wang, and S. Jha, "Towards automatic generation of vulnerability-based signatures," in Proc. IEEE Symp. Security Privacy, Oakland, CA, USA, 2006, pp. 2–16.

- [15] G. Cantrill, J. Bonwick, and M. McDougall, "Dynamic instrumentation of production systems," in Proc. USENIX Annu. Tech. Conf. (USENIX ATC), Boston, MA, USA, 2004, pp. 15–28.
- [16] E. Zelikman, Y. Wu, J. Mu, and N. Goodman, "STaR: Bootstrapping reasoning with reasoning," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022.
- [17] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-augmented language model pre-training," in Proc. Int. Conf. Mach. Learn. (ICML), Virtual, 2020, pp. 3929–3938.
- [18] W. McKinney, "Data structures for statistical computing in Python," in Proc. 9th Python Sci. Conf. (SciPy), Austin, TX, USA, 2010, pp. 56–61.
- [19] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *Knowl. Eng. Rev.*, vol. 10, no. 2, pp. 115–152, Jun. 1995.